Efficient Use of Misha

Ping Luo

Center for Neurocomputation and Machine Intelligence

Wu Tsai Institute

Fall 2026

Agenda

- Cluster Misha overview
- Manage disk space
- Understand SLURM options
- Understand QoS-based GPU scheduling
- Checkpoint and restart
- Code Development Environments
- Monitor job performance

Misha Overview

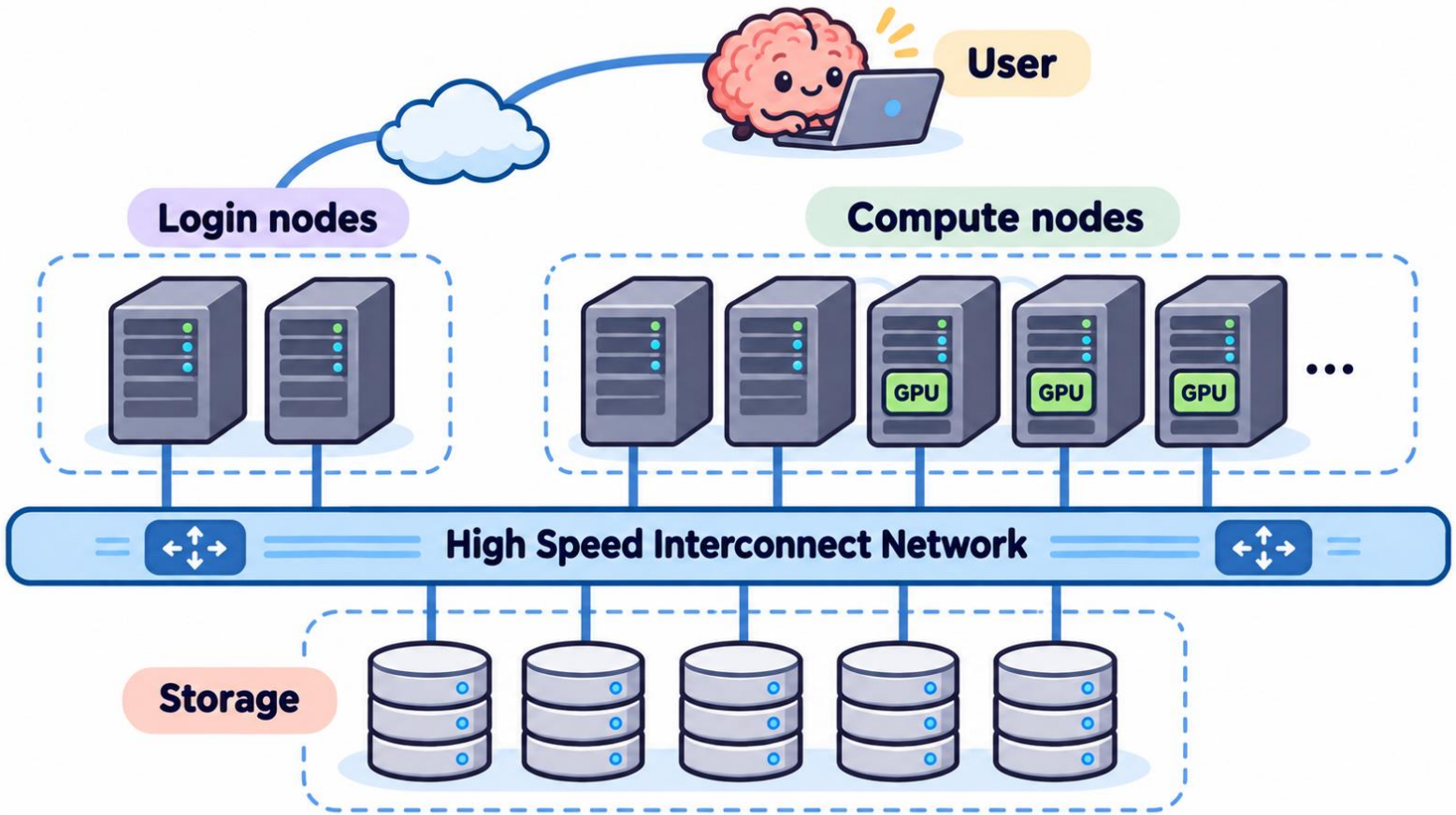
Cluster Misha



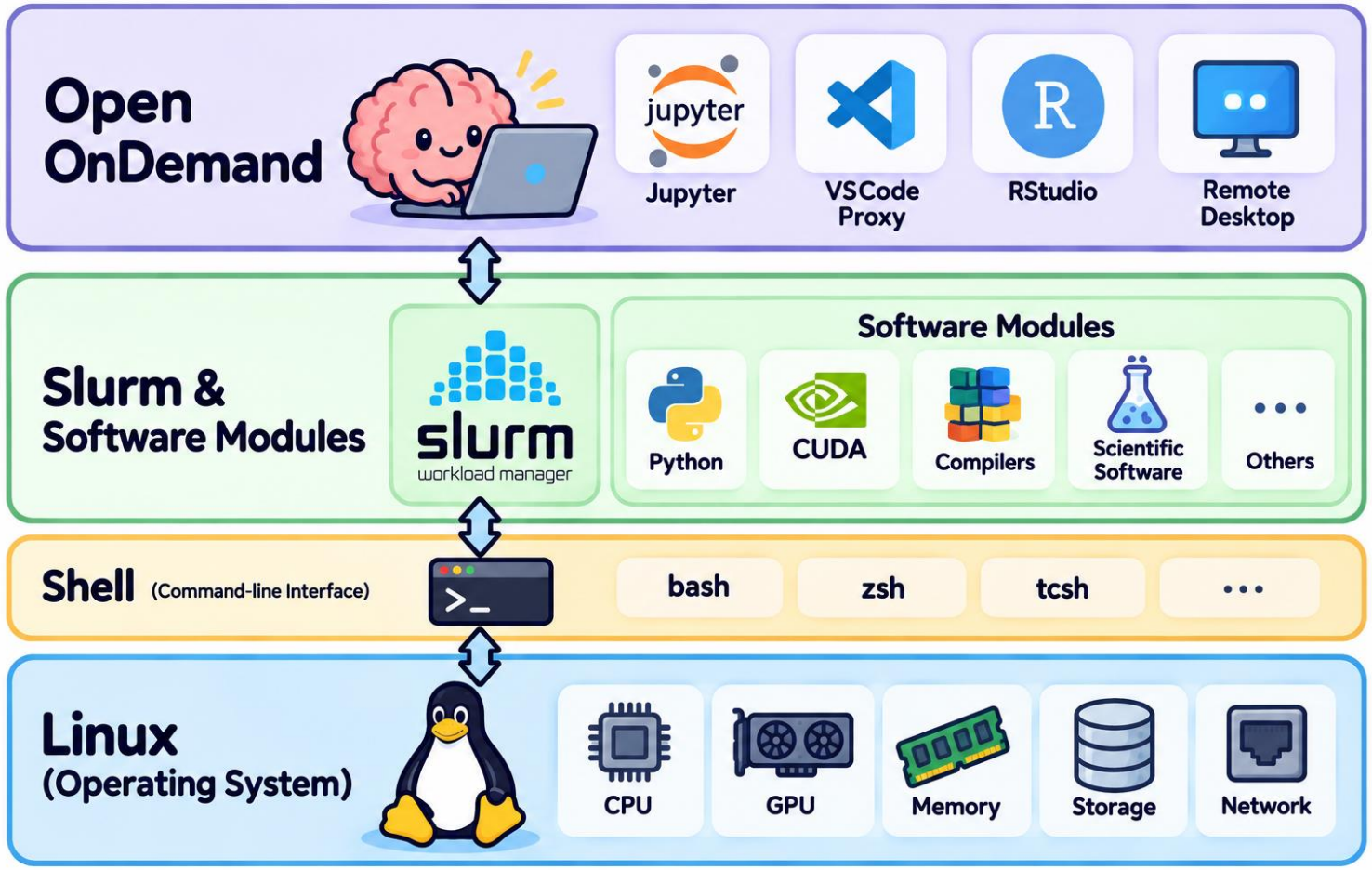
- Named in honor of Misha Mahowald
- Pioneer in brain-inspired computing
- Inventor of the first silicon retina



Cluster in a Nutshell



Cluster Software Environment

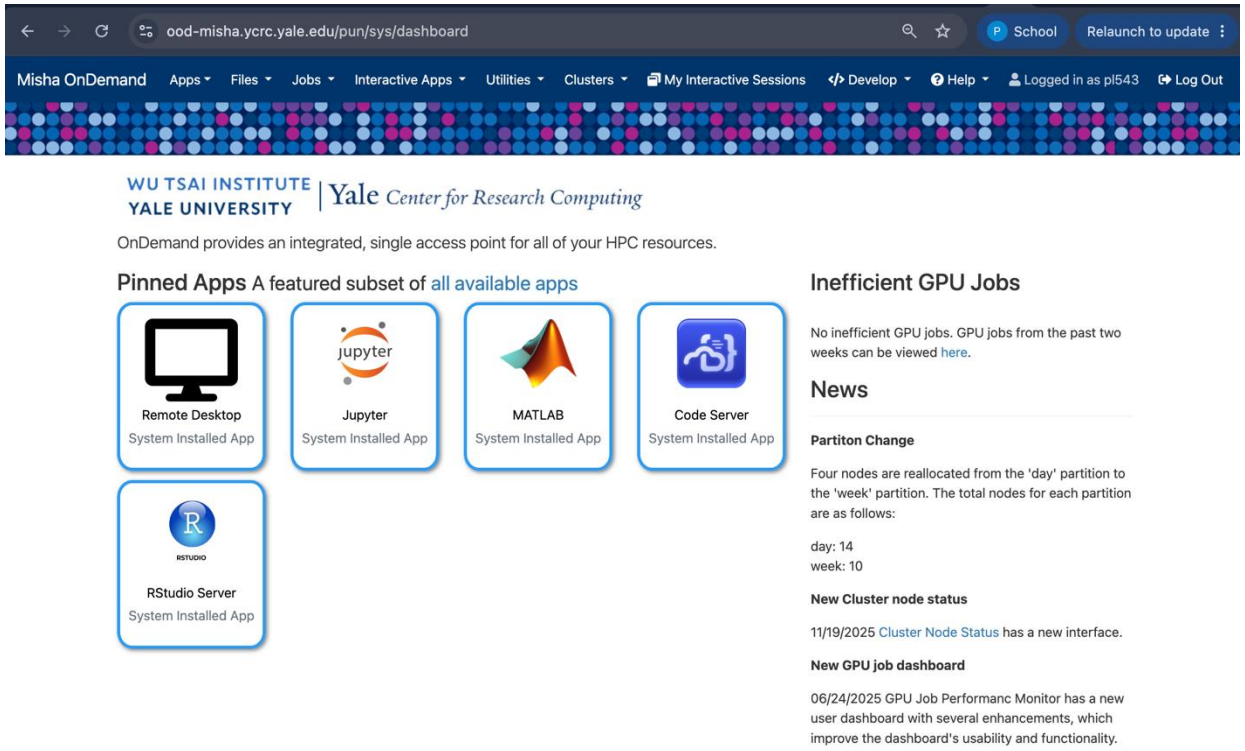


Apply for a Misha Account

- Two types of requests
  - PI request is approved by WTI
  - Researcher request is approved by PI and WTI
- Two forms are required for account application
  - YCRC Account Request Form
  - Misha Research Description Form

Misha OnDemand Portal

ood-misha.ycrc.yale.edu



- File explorer
- Job composer
- Shell access
- Interactive apps
- Utilities

Manage Storage

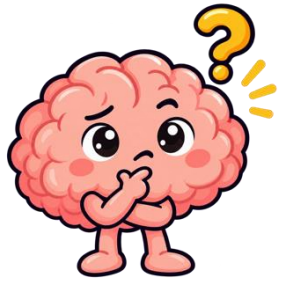
Misha Storage

Name	Location	Quota	File Limit	Type	Increase	Backup	Purged
home	/gpfs/radev/home/netid	125 GiB	500,000	user	no	yes	no
project	/gpfs/radev/project/group/netid	4 TiB	5,000,000	group	no	no	no
scratch	/gpfs/radev/scratch/group/netid	10 TiB	15,000,000	group	maybe	no	60 days
pi_group	/gpfs/radev/pi/group/netid /gpfs/marilyn/pi/group/netid	varied	varied	group	yes	no	no

Storage for NDP's PI filesets

Marilyn





Radev



Everything else

Need more storage? Complete the WTI Storage Request Form.

Monitor Your Disk Usage

getquota

ls for sous (as of Oct 15 2026 08:00)

User	Usage (GiB)	File Count
hs2272	317	255,916
jks79	0	1
mzl7	164	497,839
yd456	945	177,167
zl664	334	403,485
zss8	3	18,869
---		
hs2272	0	3
jks79	0	1
mzl7	0	22,289
yd456	1938	7,713,467
zl664	0	335
zss8	0	1,160

Quota Summary for sous (as of Oct 15 2026 08:00)

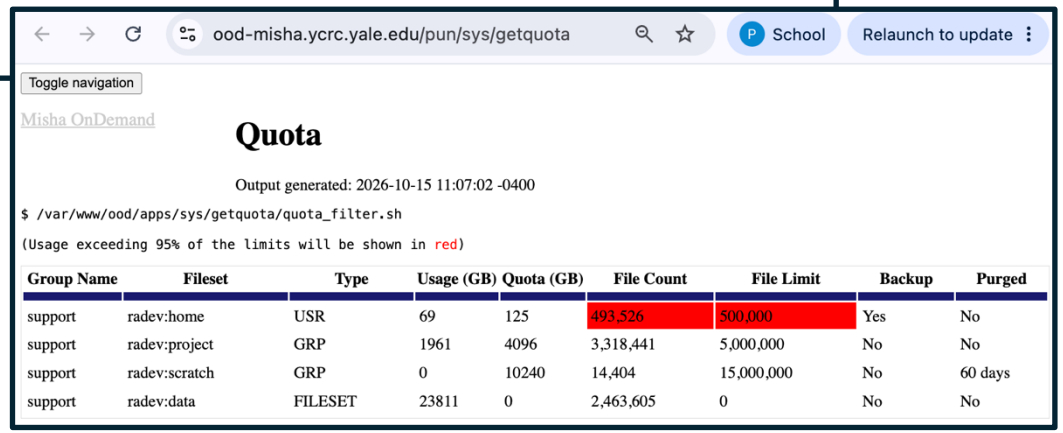
Fileset	Type	Usage (GiB)	Quota (GiB)	File Count	File Limit	Backup	Purged
radev:home	USR	3	125	8,445	500,000	Yes	No
radev:project	GRP	1762	4096	1,353,279	5,000,000	No	No
radev:scratch	GRP	1938	10240	7,737,257	15,000,000	No	60 days



Misha OnDemand

-> Utilities

-> getquota



Manage Your Home Directory

- 125 GiB space
- 500,000 file limit
- No increase



Manage Your Home Directory

- 125 GiB space
- 500,000 file limit
- No increase



- Both can cause a failure to
  - Run jobs
  - launch applications
  - Start new OOD sessions

Manage Your Home Directory

- Quota issues in home are usually caused by two applications




```
# Move conda to project or a PI fileset
# Do it with tmux or nohup
nohup mv $HOME/.conda $PROJECT/conda
# Link it back to the home directory
ln -s $PROJECT/conda $HOME/.conda
```

OPEN OnDemand

```
# find files that are older than 30 days and delete them
OODAPP_PATH=$HOME/ondemand/data/sys/dashboard/batch_connect/sys
find $OODAPP_PATH -mtime +30 -print -exec rm -rf {} \;
```

Understand Slurm Options

Slurm Job

- A Slurm job requests computing resources to run workloads on a cluster: CPUs, GPUs, memory, etc.

```
#!/bin/bash
#SBATCH --job-name=simple_test       # Name of the job in the queue
#SBATCH --output=result_%j.log       # Standard output file (%j will be replaced by the unique Job ID)
#SBATCH --nodes=1                    # Request exactly 1 physical node
#SBATCH --ntasks=1                   # Run a single task/process
#SBATCH --cpus-per-task=1            # Allocate 1 CPU core for that task
#SBATCH --mem=1G                     # Request 1 Gigabyte of memory
#SBATCH --time=00:05:00              # Time limit (Format: HH:MM:SS) - 5 minutes

# --- Print some basic diagnostics to the output log ---
echo "Job started on: $(date)"
echo "Running on node: $SLURM_NODENAME"

# --- Execute your program or commands ---
# Example: Print a simple greeting and pause for 10 seconds
echo "Hello from the Misha cluster!"
sleep 10

echo "Job finished on: $(date)"
```

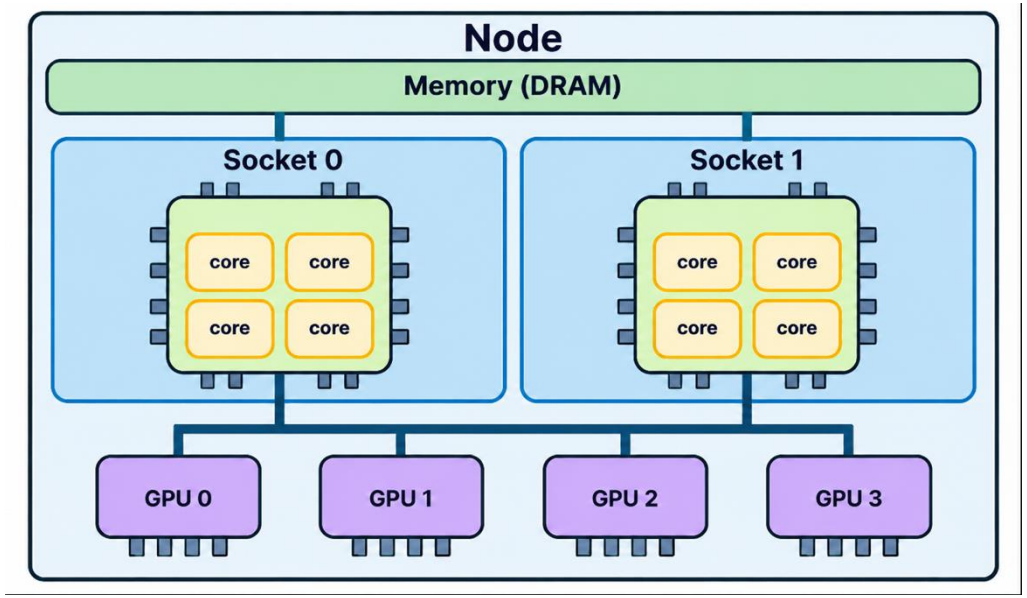
GPU Requests

- GRES (Generic Resources)
  - Special hardware resources that Slurm can allocate to jobs
  - Specify GPUs per node: `--gres=gpu:h100:2`
- TRES (Trackable Resources)
  - Resources that Slurm tracks for allocation, accounting, and scheduling
  - Specify total GPUs per job: `--gpus=h100:2`
- Other options
  - `--gpus-per-task=h100:2`
  - `--gpus-per-node=h100:2`

Memory, CPU, and GPU Ratios

- Each GPU node on Misha has
  - ~ 1TB memory
  - 32 - 48 CPU cores
  - 4 GPUs
- Recommended resource requests **per GPU**:
  - CPU cores: No more than ¼ of the node's total (8 - 12 cores)
  - Memory: Aim for no more than ¼ of the node's total (~250 GB), unless the application requires more.
- Request only the CPU cores and memory your application needs.
- Goal: Prevent GPUs from being stranded by CPU or memory exhaustion.

Resource Request and Placement Options



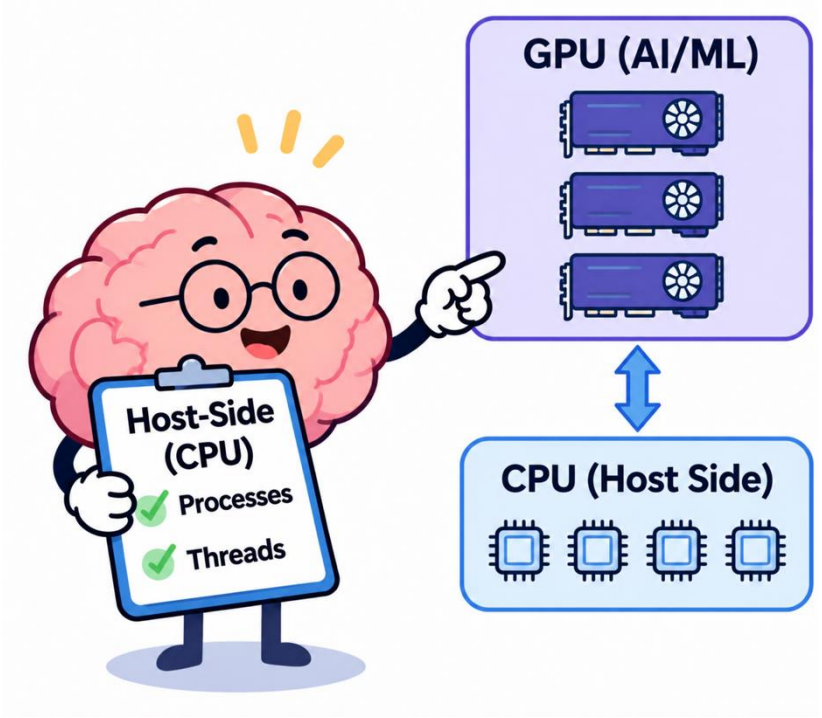
Simplified GPU node hardware topology

- In SLURM, a **core** refers to a physical CPU core, and a **CPU** refers to a schedulable processing unit.
  - A SLURM CPU is not the same as a physical multi-core CPU.
  - On Misha, a SLURM CPU corresponds to one physical CPU core on the motherboard.

On Misha, a SLURM CPU is equivalent to a physical core

Category	Options
Memory	--mem=<size>[K|M|G|T] --mem-per-cpu=<size>[K|M|G|T] --mem-per-gpu=<size>[K|M|G|T]
Tasks	--ntasks=<ntasks>, -n <ntasks> --ntasks-per-node=<ntasks> --ntasks-per-core=<ntasks> --ntasks-per-gpu=<ntasks>
GPUs	--gpus=[type:]<ngpus> --gres=gpu:[type:]<ngpus> --gpus-per-task=[type:]<ngpus> --gpus-per-node=[type:]<ngpus>
CPUs	--cpus-per-task=<ncpus>, -c <ncpus> --cpus-per-gpu=<ncpus>
Socket	--cores-per-socket=<number> --gpus-per-socket=[type:]<number> --ntasks-per-socket=<number> --sockets-per-node=<number>

Parallel Workloads in AI/ML - Host-side Execution

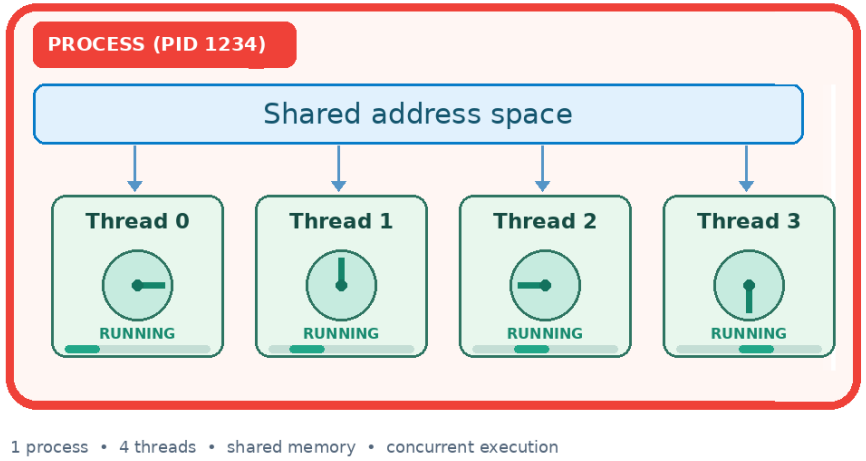


- GPU workloads require host-side processing and coordination.
- **Our focus:** CPU processes and threads, not GPU-side parallelism.
- Understanding multithreaded and multi-process workloads helps optimize Slurm resource requests.
- **Remember:** More CPUs or GPUs do not automatically make an application parallel!

Multi-threaded Workload

- Runs multiple threads within a single process on one compute node.
- Threads share the same memory address space.
- Programming model: **OpenMP**
- AI/ML example: **PyTorch CPU tensor operations**

--nodes=1 --ntasks-per-node=1 --cpus-per-task=4



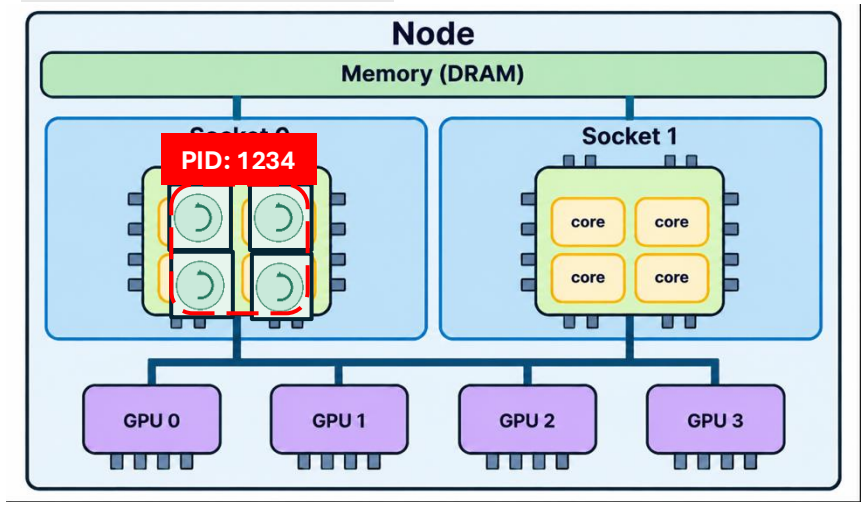
- Slrum will determine the placement of the threads.
- Set environment variables to control OpenMP thread placement and binding

```
export OMP_NUM_THREADS=4
export OMP_PLACES=cores
export OMP_PROC_BIND=spread
```

Multi-threaded Workload

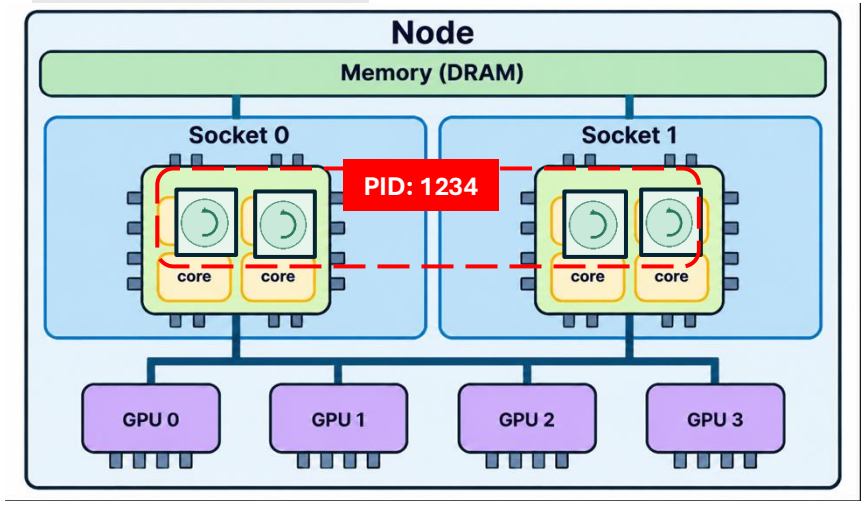
- You can control where the threads are placed.

Placement 1



--nodes=1
--sockets-per-node=1
--cores-per-socket=4
--ntasks=1
--cpus-per-task=4

Placement 2

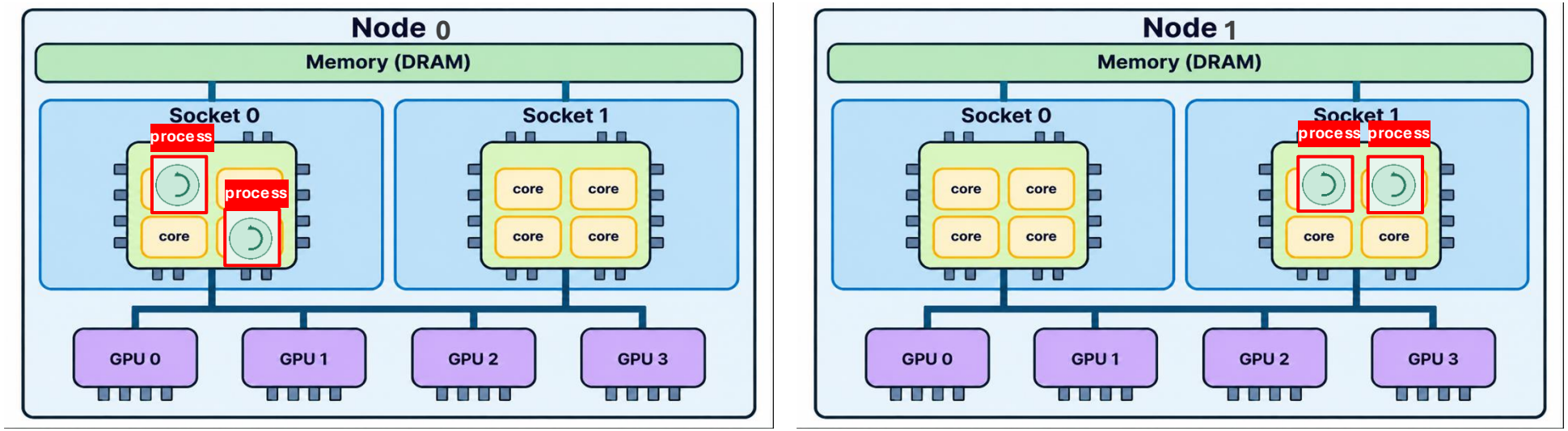


--nodes=1
--sockets-per-node=2
--ntasks-per-socket=1
--ntasks=2
--cpus-per-task=2

Multi-process Workload

- Runs multiple processes on one or multiple compute nodes.
- Each process has its own memory address space.
- Programming model: **MPI**
- AI/ML example: **PyTorch Distributed Data Parallel**

--nodes=2 --ntasks-per-node=2 --cpus-per-task=1

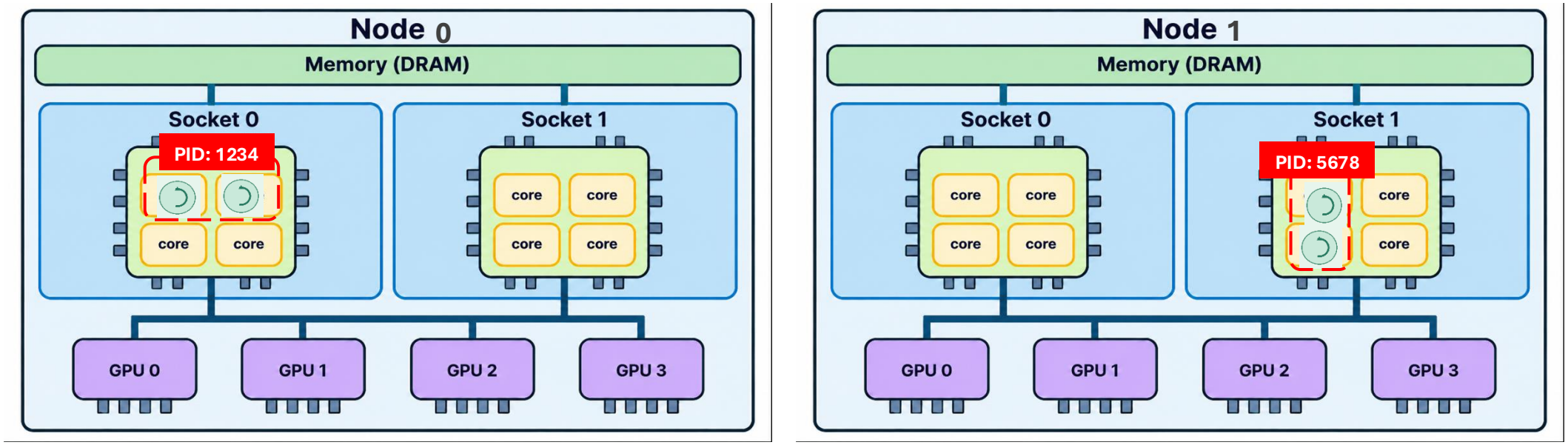


A parallel workload launched with 4 processes on two nodes, each node with 2 processes

Hybrid Workload

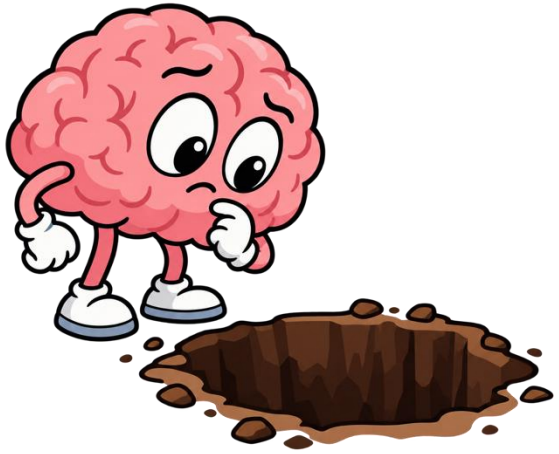
- Runs multiple processes, each with multiple threads.
- Combines distributed-memory and shared-memory parallelism.
- Programming model: **MPI + OpenMP**
- AI/ML example: **PyTorch DDP with multithreaded CPU operations**

--nodes=2 --ntasks-per-node=1 --cpus-per-task=2



A parallel workload launched with 2 multi-threaded processes on two nodes, each process with 2 threads

Common Pitfalls



- Incorrect combination of tasks, CPUs, and cores
  - Easy to detect with a good understanding of parallel workload models
- Incorrect memory or GPU requests
  - These are less obvious and easier to ignore.

Incorrect Memory/GPU Request Examples

Example 3 --mem-per-cpu=64g --ntasks-per-node=2 --cpus-per-task=8	**Intended memory:** 64G per node **Allocated memory:** 1024G	**How to fix:** --mem=64g or --mem-per-cpu=4g
Example 4 --mem-per-cpu=64g --cpus-per-gpu=8 --gpus = 1	**Intended memory:** 64G per node **Allocated memory:** 512G	**How to fix:** --mem=64g or --mem-per-gpu=64g
Example 5 --nodes=2 --gpus=2	**Intended GPU placement:** 1 per node **Actual GPU placement:** node1: 2; node: 0	**How to fix:** --gres=gpu:1 or --gpus-per-node=1

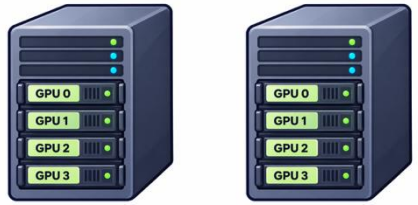
QoS-based GPU Scheduling

Motivation

PI **Alice**



PI **Bob**



PI **Carl**





Partition Alice

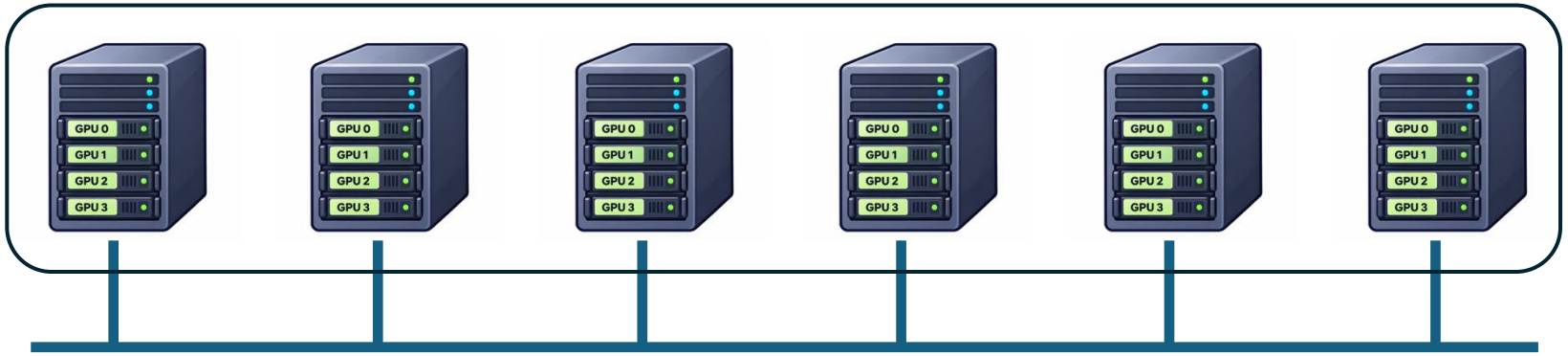
Partition Bob

Partition Carl

How to design a scheduling paradigm

- Reduce number of small partitions
- Allow each PI to have priority access to the same number of GPUs

QoS-based GPU Scheduling



partition_abc

- One partition includes all the GPU nodes.
- Each PI group has a dedicated QoS specifying the number of GPUs for the group

  qos_alice: 4 GPUs
  qos_bob: 8 GPUs
  qos_carl: 12 GPUs

- The GPU location doesn't matter.
- This makes job scheduling more flexible.

QoS and GPU Access

- Group QoS **(qos_groupname)**
  - Guarantees access to a specified number and type of GPUs.
  - GPUs can come from any eligible node.
  - Jobs wait when the group's guaranteed GPUs are already in use.
  - Must specify GPU count and type: `--qos=qos_lafferty --gres=gpu:h100:2`
- CNMI QoS **(qos_nmi)**
  - Provides access to CNMI's specified number and types of GPUs.
  - Available to all Misha users.
  - Must specify GPU count and type: `--qos=qos_nmi --gres=gpu:h100:2`
- Regular Jobs **(No QoS Specified)**
  - Can use any available GPUs on Misha.

QoS Priority and Preemption

	Group QoS	CNMI QoS	Regular Jobs
Scheduling priority	Guaranteed access	Higher priority	Standard priority
Can preempt?	Regular jobs	No	No
Can be preempted?	No	No	Yes

Preemption rules

- Group QoS jobs can preempt regular jobs to reclaim GPUs.
- Preempted jobs get a one-hour grace period to checkpoint and clean up.
  - See next section on how to do this properly.

Checkpoint and Restart

Preemption and Signal Handling

- Misha preemption is configured with a one-hour grace period.
  - A job receives SIGCONT and SIGTERM immediately when it is selected for preemption.
    - These signals go to **job steps only**, not the batch shell.
  - New end time = min{current_time+1hour, current_end_time}
  - When the new end time arrives, Slurm sends SIGCONT, SIGTERM, and SIGKILL to terminate the job.
- Signal handling is commonly used to trigger checkpointing before the job terminates.
  - This will be handled differently for batch shell and job step.

Handle Signals in User Code

A Python example

```
import signal
import time
import sys

terminate_requested = False

def signal_handler(signum, frame):
    global terminate_requested
    terminate_requested = True

signal.signal(signal.SIGTERM, signal_handler)

def checkpoint():
    print("Saving checkpoint...", flush=True)
    # Save model state, optimizer state, etc.
    time.sleep(2)
    print("Checkpoint saved.", flush=True)

print("Python program started.", flush=True)
for i in range(1000):
    if terminate_requested:
        checkpoint()
        sys.exit(0)

    print(f"Working... iteration {i}", flush=True)
    time.sleep(1)
print(”Program finished successfully.”, flush=True)
```

Handle Preemption in Batch Shell

```
#!/bin/bash
#SBATCH -p gpu
#SBATCH --gres=gpu:a100:1
#SBATCH -n 1 --ntasks-per-node=1
# Send a SIGTERM 1 hour before end time
#SBATCH --signal=B:TERM@3600

# Signal handler function
signal_handler() {
  echo "Signal received in BATCH script.
  # Forward SIGTERM to Python
  kill -TERM "$python_pid”
}

trap ‘signal_handler’ SIGTERM
python -u my_program.py &
python_pid=$!

wait “$python_pid”
```

How it works:

- A job receives SIGTERM immediately when it is selected for preemption
  - The signal goes to the job steps, not the batch shell. It is ignored since no one is there to catch the signal.
- At the same time, its end time is updated: new_end_time = now + 1 hour
- New end time - 1 hour = now, this triggers a SIGTERM to the batch shell.
- The shell traps the signal and calls the signal handler.
- The signal handler sends the child process running the user application a SIGTERM, which triggers checkpointing in the user code.

Handle Preemption in Job Step

```
#!/bin/bash
#SBATCH -p gpu
#SBATCH --gres=gpu:a100:1
#SBATCH -n 1 --ntasks-per-node=1


srun python -u my_program.py
```

How it works:

- A job receives SIGTERM immediately when it is selected for preemption.
- The signal is sent to all job steps.
- The process running the user application receives a SIGTERM.
- The signal triggers checkpointing in the user code.

Automatic Checkpoint-Restart

How it works:

- When a job is preempted, the application saves a checkpoint and performs cleanup.
- The job is requeued and waits for resources.
- When resources become available, Slurm restarts the job.
- The application loads the latest checkpoint and resumes execution.

Two final steps to enable automatic restart:

- **In your application:** Add code to load the latest valid checkpoint at startup.
- **In your batch script:** Add #SBATCH --requeue to allow the job to be requeued.

```
#!/bin/bash
#SBATCH -p gpu --gres=gpu:a100:1 -n 1 --ntasks-per-node=1
#SBATCH --requeue

export PYTHONUNBUFFERED=TRUE
[ "x${SLURM_RESTART_COUNT}" == "x" ] && SLURM_RESTART_COUNT=0
srun ./my_program.py 2>&1 > out.${SLURM_JOB_ID}.${SLURM_RESTART_COUNT}
```

Code Development Environments

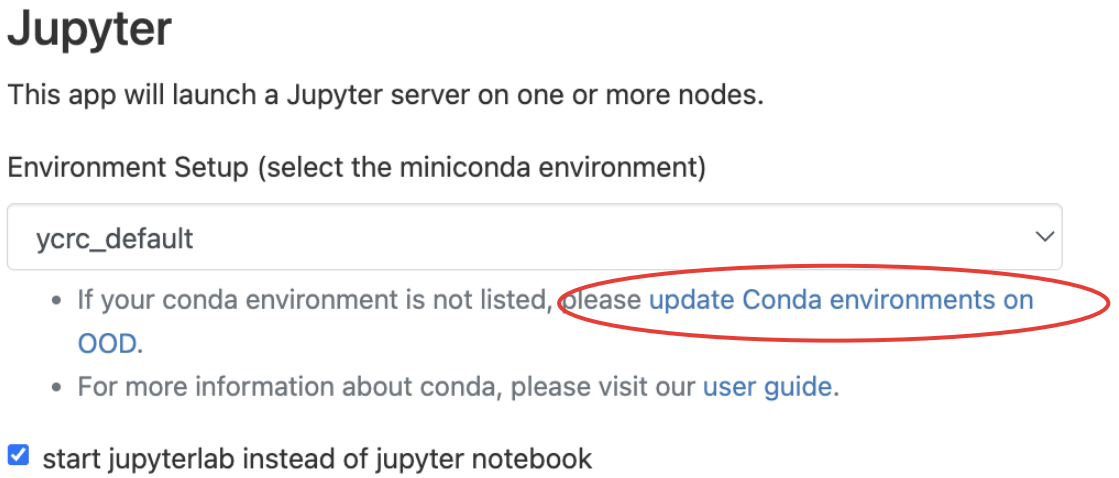
Jupyter Development Environment

- Jupyter Notebook and JupyterLab
  - Interactive, browser-based environments for writing and running code, visualizing data, and documenting results.
  - Widely used for data analysis, scientific computing, and AI/ML development.
- Misha OnDemand Jupyter App
  - Runs a Jupyter server on a Misha compute node.
  - The user connects to the server and opens Jupyter Notebook or JupyterLab through their local web browser.
  - Requires a Conda environment with Jupyter installed.
  - Supports selecting from multiple Conda environments.

Add a Conda Environment for Jupyter App

- Create a Conda environment for Jupyter.
  - Make sure “jupyter” is installed.
  - This installs the Python kernel by default.
  - You can add kernels for other languages: R, Julia, bash, etc.
- Update the Conda env in the Misha Jupyter App.
  - Click ‘update Conda environments on OOD’ to refresh the environment list.
- Launch the Jupyter App
  - Select your new Conda environment.

```
# Do this on a compute node
salloc --mem=30g -n1 -c4
module load miniconda
conda create -n jupyter_env jupyter
conda activate jupyter_env
conda install -c r r-essentials r-irkernel \
bash-kernel
```



VS Code and Code Server

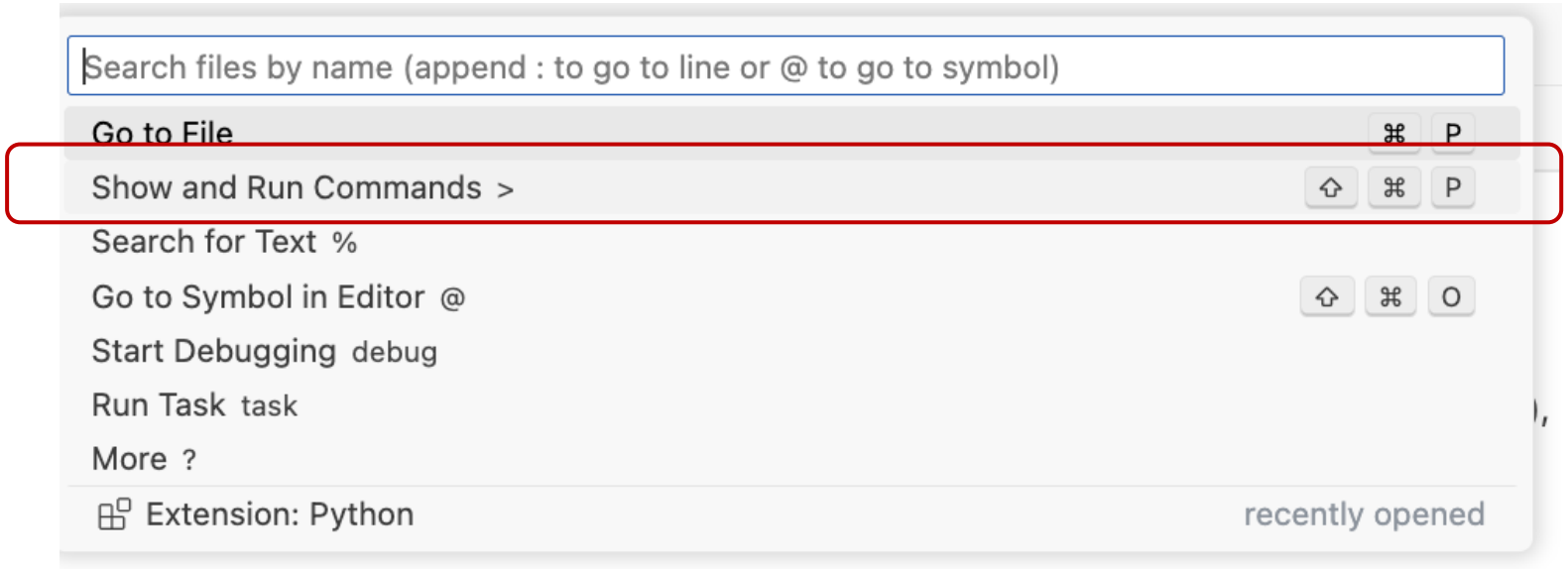
- Use VS Code for code development
  - Runs on your local machine
  - The backend server runs on a Misha compute node
  - Communication between frontend and backend:
    - **VS Code SSH extension**: use **VS Code Proxy App** to simplify access
    - **VS Code Remote Tunnels**: set up manually
- Use Code Server for code development
  - Open-source and web-based version of VS Code
  - Everything runs on a Misha compute node.
  - Users access the IDE through a web browser.
  - May lack some niche extensions VS Code has.

Integrate Jupyter with VS Code or Code Server

- Install the required extensions in VS Code:
  - Python
  - Jupyter
- Select the Python interpreter:
  - Specify **/path/to/Python** from the Jupyter Conda environment. This step is key for integration.
- Select the kernel for your notebook.

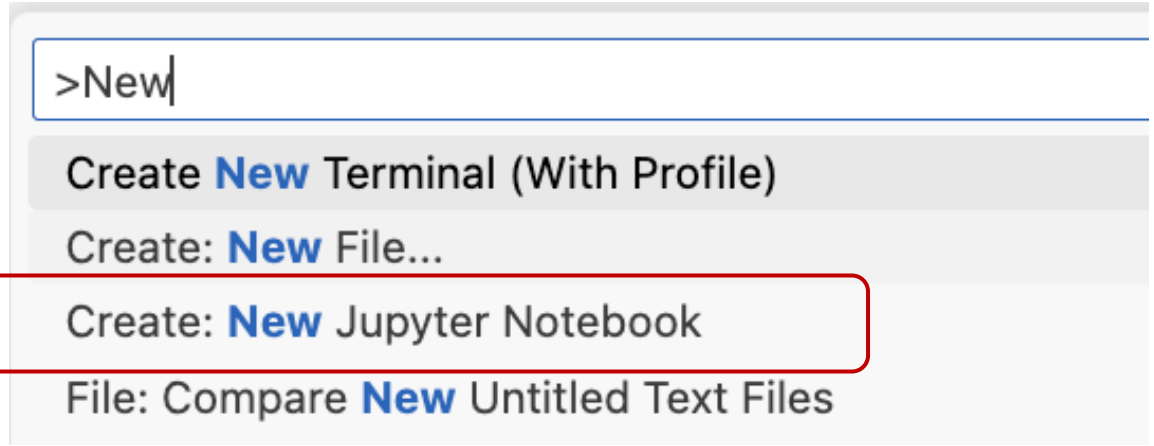
Integrate Jupyter with VS Code or Code Server

Click the top search bar -> click ‘Show and Run Commands’



Integrate Jupyter with VS Code or Code Server

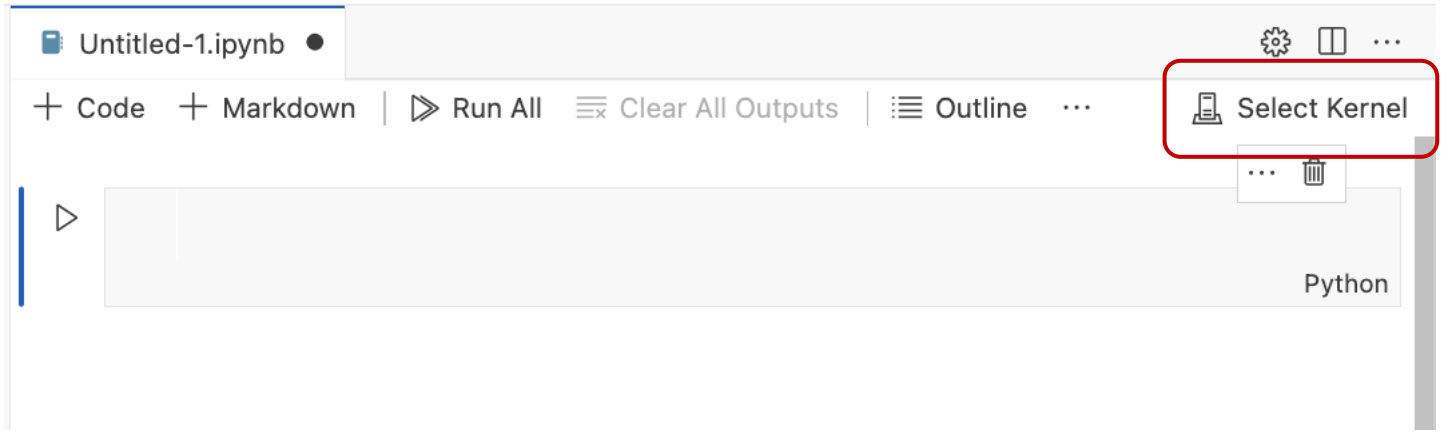
Search for ‘New Jupyter Notebook’ and click it. A new notebook will be created.



Integrate Jupyter with VS Code or Code Server

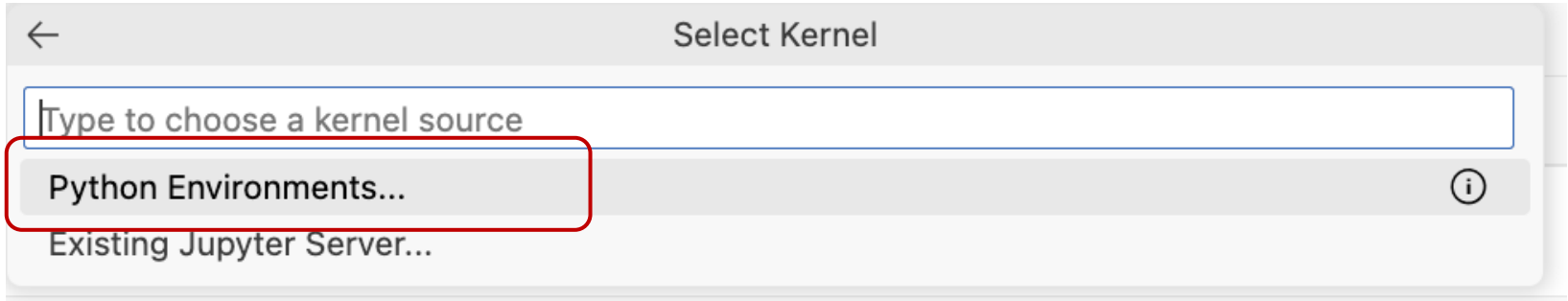
- To select the right Python, click ‘Select Kernel’ and then follow the steps.

Click ‘Select Kernel’ on the top right corner of the notebook



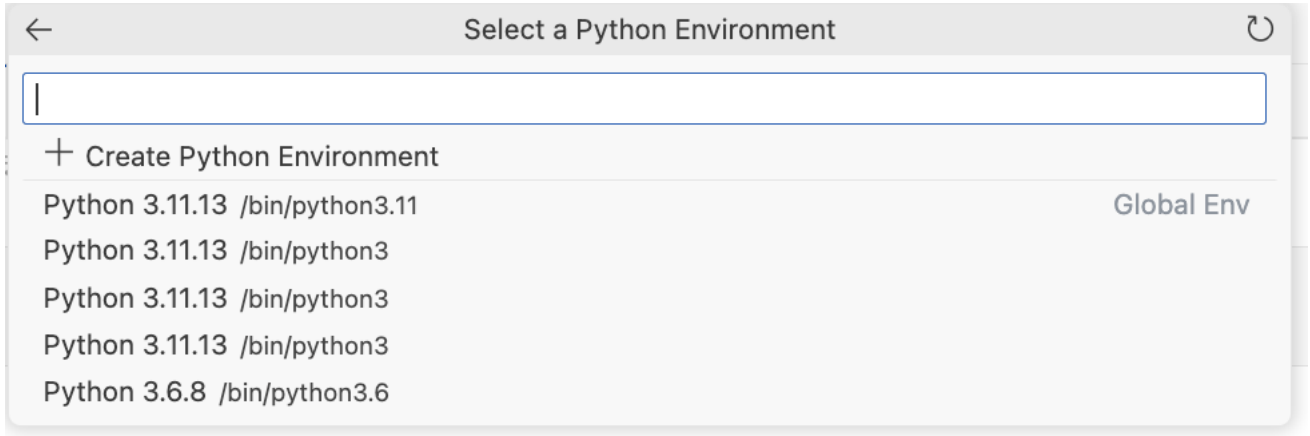
Integrate Jupyter with VS Code or Code Server

Click ‘Python Environments’ from the command center

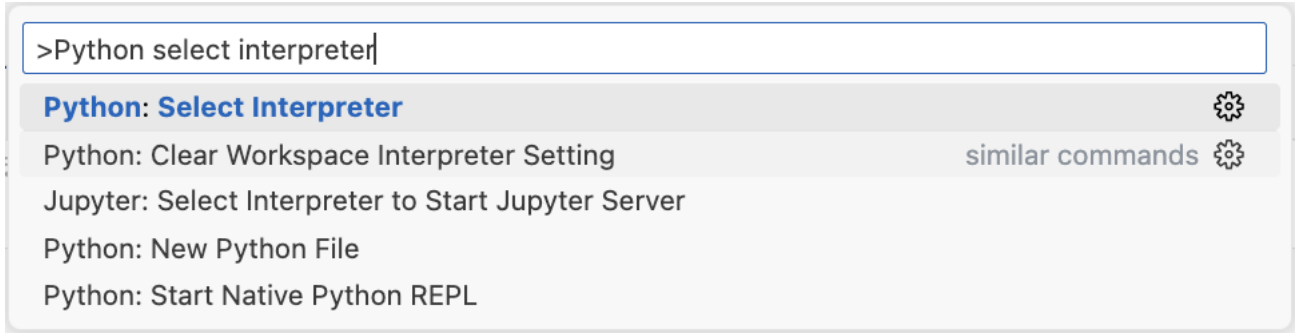


Integrate Jupyter with VS Code or Code Server

A list of Python interpreters will be provided.

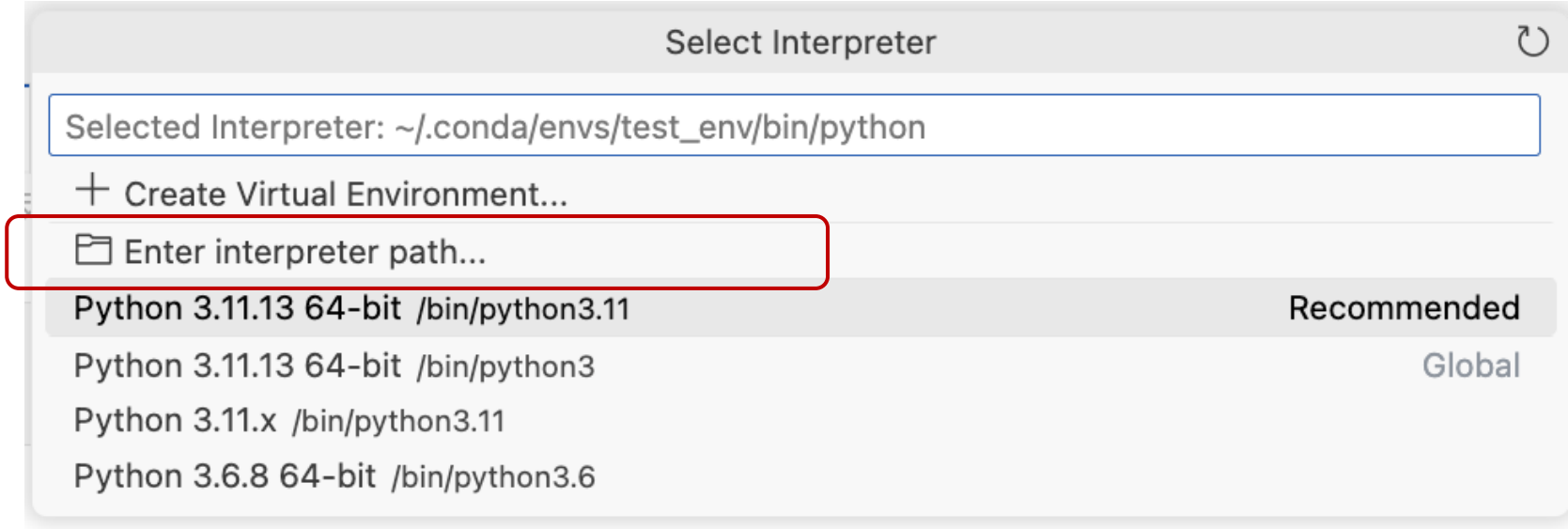


If your Python interpreter is listed, select it. If it is not listed, go back to the **command center** and search for ‘python select interpreter’

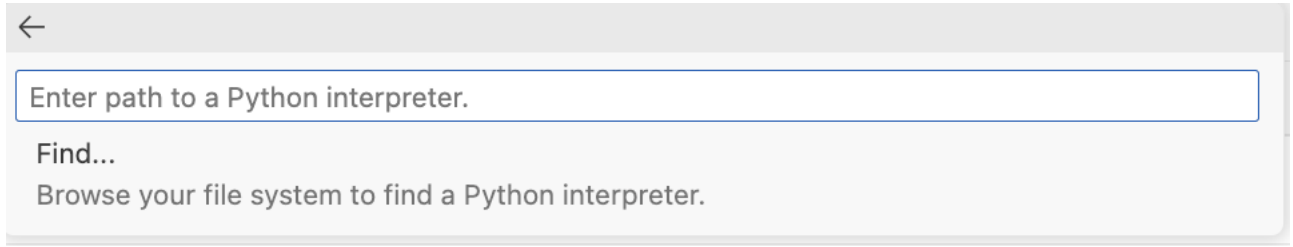


Integrate Jupyter with VS Code or Code Server

Click ‘Enter interpreter path’

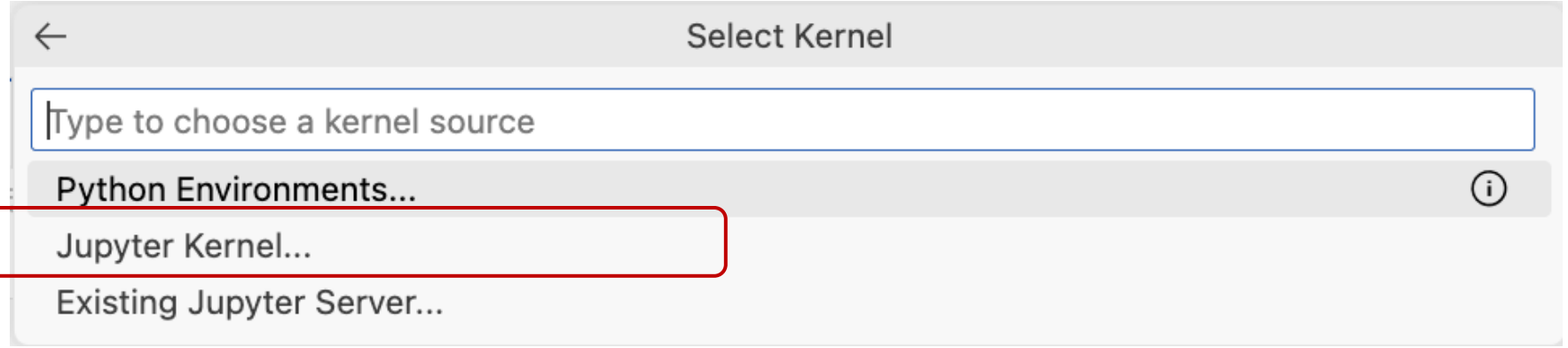


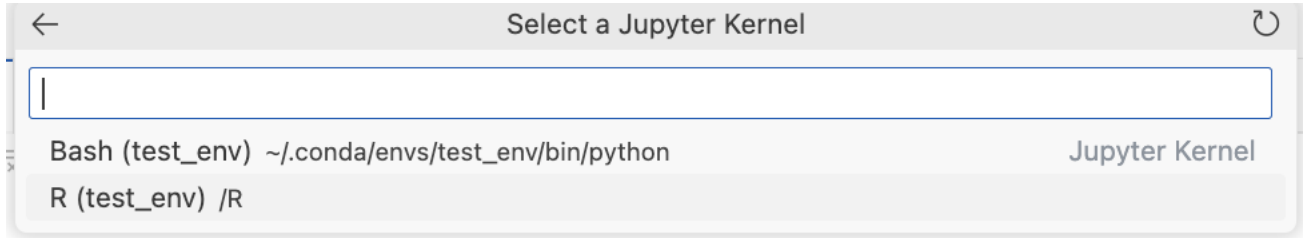
Specify the path to your Python interpreter in the **Jupyter Conda environment**.



Integrate Jupyter with VS Code or Code Server

- If no error is shown, the integration is successful.
- You can pick a different kernel:
  - Click ‘Select Kernel’ -> click ‘Jupyter Kernel’





GPU Job Performance

Node and Job Status

https://ood-misha.ycrc.yale.edu/pun/sys/cluster-status

https://ood-misha.ycrc.yale.edu/pun/sys/job-performance

Daily GPU Job Performance Alerts

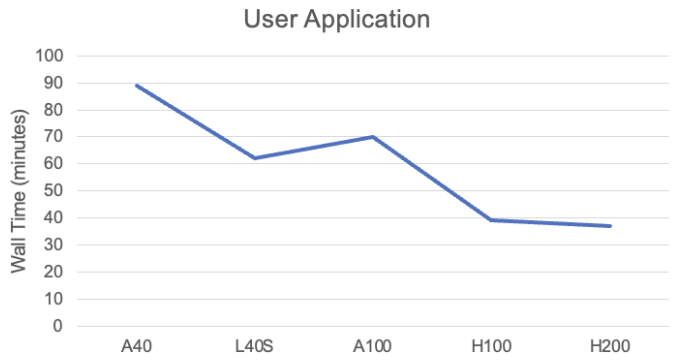
- **24/7 monitoring:** GPU utilization is continuously monitored to identify inefficient resource use.
- **Daily alerts:** Email notifications are sent at **5 PM** to users with inefficient GPU jobs.
- **Exemption:** Jobs running for less than 2 hours are excluded.
- **Criteria for Inefficient GPU Usage**
  - A GPU is considered underutilized if it remains idle for **70% or more** of the job's wall time.
  - For multi-GPU jobs, **each GPU is evaluated independently**.

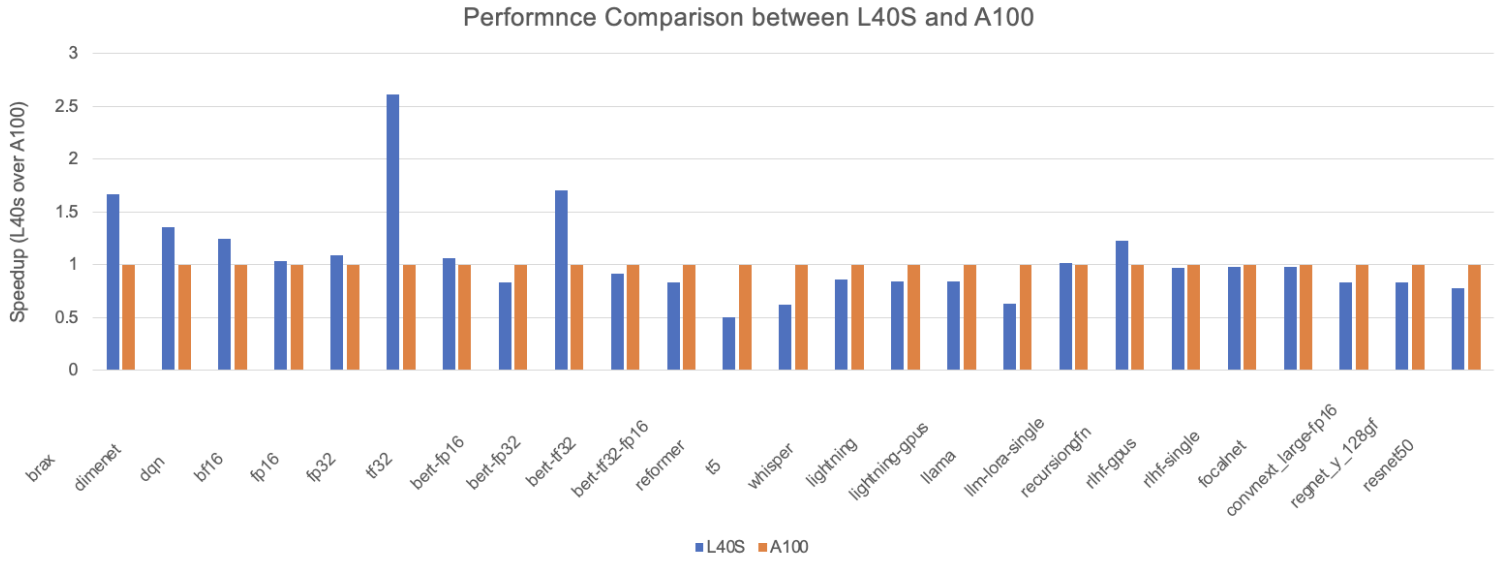
GPU Benchmarks

Takeaways

- **Low GPU utilization?** Consider optimizing your code or using a less powerful GPU.
- **L40S vs. A100:** L40S can outperform A100 for certain workloads.
- **Think beyond high-end GPUs:** A40 and L40S nodes are often more available, potentially reducing job wait times.







Get Help

- Email:
  - hpc@yale.edu
  - ping.luo@yale.edu
- In-person consultation:
  - 100 College Street, Rm 1160

Useful Links

- Misha user guide
- Misha account application
- Misha storage application
- YCRC user guide